

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	M.KOUSHIK REDDY	Reg. No.:	192372185
Section / Batch:	CSE-AI	Date:	03/08/2026

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

Code of Conduct

I M.Koushik Kumar Reddy /192372185 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	

5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator Faculty Incharge:-

QUESTIONS

1. A computer system uses 8-bit 2's complement representation to process signed integer arithmetic. Consider two numbers: -52 and $+19$. Analyze in detail how the processor performs both addition and subtraction, including binary conversion, sign extension, intermediate steps, and the final result. Determine whether overflow occurs in each operation and explain how the processor detects and handles overflow.
2. A modern processor is designed with an 8-bit carry look-ahead adder to accelerate arithmetic operations. Given two binary operands, analyze how the generate and propagate signals are produced, how carry bits are calculated simultaneously, and compare the execution time of both adders. Justify why CLA is preferred in high-performance processors.
3. A digital processor performs signed multiplication using Booth's algorithm for efficient arithmetic computation. Multiply $+11$ and -7 , and analyze each iteration of the algorithm by showing the contents of the accumulator, multiplier register, Booth bit, and the arithmetic right shifts. Explain how Booth's algorithm minimizes the number of addition and subtraction operations while correctly handling signed operands.
4. A binary arithmetic unit supports both restoring and non-restoring division techniques for integer division. Analyze the execution of the division of $25 \div 4$ using both methods by showing the intermediate remainder values, quotient generation, and register contents after each step. Compare the algorithms based on execution time, hardware complexity, and computational efficiency, and explain why one method is preferred over the other.
5. A floating-point unit (FPU) processes real numbers according to the IEEE 754 standard. Analyze the representation of the decimal number $+26.75$ in both single-precision (32-bit) and double-precision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, analyze the floating-point addition of 26.75 and -8.5 , explaining exponent alignment, mantissa operations, normalization, rounding, and the effect of precision on the final result.

ANSWERS

1.

① Given ;
 $\{ \text{8-Bit 2's Complement} \}$
 Two numbers : $[-52, +19]$

② Addition : $-52 + 19$

Binary $+19$: 00010011
 Binary -52 : -0110100 $\rightarrow$ 1's
 11001011 $\rightarrow$ 2's
 +1

Now add :- $+52$:- $\underline{11001100}$

$\Rightarrow$ $\begin{array}{r} 11001100 \\ 00010011 \\ \hline 11011111 \end{array} \Rightarrow -33$

- Overflow :- NO
- Reason :- Operands have different signs, So cannot overflow occur.

③ Subtraction : $-52 - 19$

- Convert $+19$ to -19

00010011 $\rightarrow$ 011101100 $\xrightarrow{\text{2's comp}}$ $\boxed{11101101}$

2.

Now add :- $\begin{array}{r} 11001100 \\ 11101101 \\ \hline 10111001 \end{array} \Rightarrow -71$

- Overflow :- NO
- Detection :- Overflow occurs only when adding two numbers of the same sign gives the opposite sign.

* *

② Carry Look - Ahead Adder (CLA)

Note :- generate and propagate signals

$\rightarrow$ For each bit :-

- Generate :- $g = A \times B$
- Propagate :- $p = A \oplus B$

$\rightarrow$ Carry Equations :-

- $C_1 = g_0 + P_0 C_0$
- $C_2 = g_1 + P_1 g_0 + P_1 P_0 C_0$

$\rightarrow$ Comparison :-

Ripple Carry Adder	Carry Look-Ahead Adder
① Carry propagates sequentially	① Carry computed simultaneously
② Slower	② Faster
③ Simple Hardware	③ More Hardware

3.

Example :- (4-bit) :- $A = 1101$
 $B = 0111$

i	A_i	B_i	g_i	P_i
0	1	1	1	0
1	0	1	0	1
2	1	1	1	0
3	1	0	0	1

*


③ Booth's Algorithm

Given : 5-bit representation

- Multiplicand (M) $\rightarrow +11 = 01011$
- $-M \rightarrow 10101$
- Multiplier (Q) $\rightarrow (-7) = 11001$
- A $\rightarrow 00000$
- Q_{-1} (Q_0 / Booth bit) $\rightarrow 0$
- n $\rightarrow 5$

Table :-

5-bit representation

- M (Multiplicand) = $+11 = 01011$
- $-M$ (2's complement of M) = $-11 = 10101$
- Q (Multiplier) = $-7 = 11001$
- A (Accumulator) = 00000
- $Q_{-1} = 0$
- n (No. of bits) = 5

Rules for Action
(Based on Q_0 and Q_{-1})

Q_0	Q_{-1}	Action
0	0	No operation
0	1	$A = A + M$
1	0	$A = A - M$
1	1	No operation

Step (i)	A (5-bit)	Q (5-bit)	Q_0	Q_{-1}	Action	Result in A	Arithmetic Right Shift (A, Q, Q_{-1})
0	00000	11001	1	0	Initial	00000	—
1	00000	11001	1	0	$A = A - M$	10101	11010 1 1
2	11010	01100	0	1	$A = A + M$	00001	00000 1 0
3	00000	10110	0	0	No operation	00000	00000 0 1
4	00000	01011	1	0	$A = A - M$	10101	11010 1 1
5	11010	10101	1	1	No operation	11010	11101 0 1
6	11101	01010	0	1	$A = A + M$	00100	00010 0 0
7	00010	00101	1	0	$A = A - M$	10111	11011 1 0
8	11011	10010	0	0	No operation	11011	11101 1 1
9	11101	11001	1	1	No operation	11101	11110 1 1
10	11110	11100	0	1	$A = A + M$	00001	00001 1 0

After $n = 5$ steps, final (A, Q) = 00001 10011

$\Rightarrow \underline{AQ = 0000110011}$

- AQ (Final) = 0000110011
- MSB (Leftmost bit) = 0 $\Rightarrow$ Number is negative.
- To find decimal value, take 2's complement.
 1's complement of 0000110011 $\Rightarrow$ 1111001100
 Add 1 $\Rightarrow$ 1111001101
- $1111001101_2 = 253_{10}$
- Since it is negative $\Rightarrow 256 - 253 = 3 \Rightarrow -77$

2's complement method

```

0000110011
1's complement
1111001100
+ 1
1111001101

```

Final Answer : $AQ = 1110110011_2 \Rightarrow -77_{10}$

4.

④ (A) RESTORING DIVISION

- Dividend = 25 = 11001_2
- Divisor = 4 = 00100_2
- No. of bit = 5

Table :-

n	A	Q	q ₀	Operation
5	00000	11001	—	Initial
5	00001	10010	—	Shift left
5	11101	10010	0	A = A - M
4	00001	10010	0	Restore A
4	00011	00100	—	Shift left
4	11111	00100	0	A = A - M
3	00011	00100	0	Restore A
3	00110	01000	—	Shift left
3	00010	01001	1	A = A - M
2	00100	10010	—	Shift left
2	00000	10011	1	A = A - M
1	00001	00110	—	Shift left
1	11101	00110	0	A = A - M
0	00001	00110	final	Restore A

Quotient (Q) = 00110_2 = 6

Remainder (A) = 00001_2 = 1

Final Answer : Quotient = 6 , Remainder = 1

⑤ (B) NON - RESTORING DIVISION

Algorithm :- ① Perform final correction if A is negative.

Table :-

n	A	Q	Operation
5	0000	11001	Initial
5	11101	10010	Shift , A = A - M
4	00000	00100	Shift , A = A + M
3	11100	01000	Shift , A = A - M
2	00001	10001	Shift , A = A + M
1	11111	00010	Shift , A = A - M
0	00001	00110	Final correction

• Quotient (Q) = 6 = 00110

• Remainder (A) = 1 = 00001

Restoring

- Restore remainder if negative.
- More operations
- Slower

Non - Restoring

- No restore step
- Fewer operations
- Faster

⑤ IEEE 754 Floating-point Representation

- So, convert to:- 1.101011×2^4 (32-bit)

(b) Double precision (64-bit)

- [illegible]

Double precision (64-bit)

[illegible]

1-bit 11-bit 52-bit

$$= 0|0000000||0|0||00000000000000000000000000000000$$

Conclusion :-

$$26.75 - 8.5 = 18.25$$

$$18.25 \text{ (decimal)} = 10010.01_2$$

(check)

IEEE 754 provides high precision, normalization and standard representation, ensuring accurate floating point arithmetic.